Real Time Air Quality Monitoring System

Ambulkar Madhusudan
Dept. of Computer Science
Frankfurt UAS
Frankfurt am Main, Germany
madhusudan.ambulkar@stud.fra-uas.de

Khan Sahil
Dept. of Computer Science
Frankfurt UAS
Frankfurt am Main, Germany
sahil.khan@stud.fra-uas.de

Ahmed Wahaj
Dept. of Computer Science
Frankfurt UAS
Frankfurt am Main, Germany
wahaj.ahmed@stud.fra-uas.de

***Abstract*—The monitoring of indoor air quality is a critical but often overlooked challenge, primarily due to standard sensors providing raw, isolated data devoid of actionable context. This paper presents a comprehensive, real-time environmental sensing system powered by Zephyr RTOS. The proposed solution is an end-to-end Internet of Things (IoT) architecture utilizing Nordic `nRF52840` edge sensing combined with serial data transfer. By incorporating sensor fusion techniques, the system captures high-fidelity data on major pollutants, including Carbon Dioxide (CO_2), Volatile Organic Compounds (VOCs), Particulate Matter (PM) and Humidity. Furthermore, it leverages a Python and Streamlit-based analytics pipeline to deliver sub-second latency visualization and dynamic alerts. The primary objective is to develop a low-cost, real-time IoT system that mitigates health risks by providing actionable, threshold-based advisories for indoor ventilation.**

***Index Terms*—Internet of Things (IoT), Air Quality Monitoring, Edge Computing, Sensor Fusion, Real-Time Operating Systems (RTOS).**

I. Introduction

Indoor air quality (IAQ) is one of the most important and systematically underestimated public health problems of the modern age. According to World Health Organization (WHO) indoor air pollution causes an estimated number of premature deaths of 3.2 million per year and prolonged exposure to high levels of Carbon Dioxide (CO_2), Particulate Matter (PM), and Volatile Organic Compounds (VOCs) has been associated with all sorts of acute and chronic diseases such as respiratory disease, cardiovascular impairment, and accelerated cognitive decline [1]. The problem is especially acute in the modern built environment, where energy-efficient construction practices have so greatly reduced natural ventilation and now allow pollutant concentrations to increase at rates much higher than outdoor ambient concentrations. Studies carried out in controlled indoor environments have shown that carbon dioxide (CO_2) levels upwards of 1000ppm are linked to measurable reductions in decision-making ability of occupants while prolonged exposure to PM2.5 concentrations above 35μg/m^3 has been directly correlated with an increase in hospital admissions [2]. Despite the well-documented severity of these risks, indoor air quality is still largely invisible to building occupants, again because there is no cost-effective, real-time indoor air quality monitoring infrastructure.

The most significant problem of modern environmental monitoring is not the sensor availability but the lack of systems that can translate the raw data that is produced by sensors into meaningful advisories that are somehow contextually aware. Conventional deployments are normally limited to such standalone hardware with data point output for isolated readings for one type of pollutant, without the ability to fuse together multi-parameter inputs, detect compound hazard conditions and output real-time alerts to occupants. Furthermore, most of the existing low-cost solutions for IAQ are operating on cloud-dependent architectures, which creates software latency, privacy concerns and single points of failure, rendering them unsuitable for safety-critical indoor environments. There is, therefore, an obvious and unfulfilled need for a system that is native to the edge and incorporates multiple sensors; one which can perform real-time fusion, hazard classification, and advisory generation; one which does not rely on outside infrastructure; a system that must also be capable of performing these functions in real-time.

To address this limitation, this paper presents the design, implementation and evaluation of a low-cost and end-to-end IoT system for real-time indoor air quality monitoring. The proposed solution is based on the Nordic `nRF52840` microcontroller running the Zephyr Real-Time Operating System (RTOS) which gives the deterministic guarantees of execution required for concurrent multi-sensor polling. A sensor fusion approach is used to aggregate high fidelity data from three specialized components that are the Sensirion `SCD41` for CO_2 and Relative Humidity (RH), the Sensirion `SPS30` for PM2.5 and PM10 and the `CCS811` for Total Volatile Organic Compounds (TVOC) and eCO_2, across a custom split-rail power topology designed to manage the mixed nature of the voltage requirements of the sensor array. On the software side, a Python-based Extract, Transform, Load (ETL) pipeline with sub-10ms ingestion latency processes the serial data stream and feeds a Streamlit dashboard capable of pushing visual updates every 1,000 milliseconds. Hazard classification is done locally based on a strict rule-based safety logic engine mapped to ASHRAE standards allowing threshold-based ventilation advisories free of any cloud dependency.

To achieve high accuracy and reliability and data integrity, the system architecture is designed in a two-layer implementation:

- Sensing and Gateway Layer: Responsible for robust data acquisition and validation using low-cost sensor nodes and a microcontroller gateway running Zephyr RTOS. This layer manages multi-sensor polling, serial data trans-

mission, and local data buffering.

- Decision-Making and Visualization Layer: Implements a deterministic Safety Logic engine that continuously evaluates fused multi-parameter sensor values against ASHRAE-defined thresholds to autonomously classify environmental severity and deliver real-time ventilation advisories to the user interface.

The main contributing factors of this work are as follows:

- The design and deployment of a mixed-voltage, multi sensor edge architecture integrating photoacoustic CO_2 sensing, laser particulate matter detection, and MOX based TVOC monitoring on a single Zephyr RTOS plat form, with a custom split-rail power topology to manage divergent sensor voltage requirements.
- The implementation of a robust, fault-tolerant ETL pipeline featuring a sub-10ms Regex-based ingestion engine and an advanced brace-counting fragmentation recovery algorithm, capable of reconstructing fragmented JSON packets from partial serial streams without fatal parsing exceptions.
- A deterministic, ASHRAE-compliant Safety Logic engine that fuses inputs from three independent pollutant streams to autonomously determine worst-case environ mental severity across OK, WARN, ALERT and CRITICAL tiers, with Green-Yellow-Orange-Red visual advisory output updated at 1,000ms intervals.
- A validated, 48-hour continuous operational stress test demonstrating long-term system stability, bounded memory utilization, and fault tolerance under realistic high frequency IoT data conditions.

II. Related Work and Background

This section provides a review of the body of prior work that is relevant to the design of the proposed system, covering four primary areas, including the development of indoor air quality monitoring architectures, paradigms of edge computing in IoT sensing, core areas of sensor fusion/mixed-voltage power topologies, the design of ETL pipelines in real-time scheduling, and the use of standardized safety thresholds underlying hazard classification for indoor air quality. Where appropriate, the limitations of previous approaches are identified to provide a motivation for specific design choices made in this work.

A. The Evolution of Indoor Air Quality Monitoring

Early indoor air quality monitoring systems were single pollutant, stand-alone hardware deployments that provided raw data with no contextual processing at all. A comprehensive review on the application of low-cost sensor technologies for air quality monitoring was done, and it was found that the major drawback with cheap sensor technology-based air quality monitoring systems was not sensor hardware sensitivity but lack of data fusion and real-time analytical capability. The survey of more than 40 commercially deployed sensor nodes saw most reporting isolated numbers of CO_2 or PM with no cross-validation between pollutant streams and reliable compound hazard detection turned practically impossible. [3].

A network of distributed low-cost sensors for the acquisition of air quality in the urban space, showing that fusion of two or more parameters was significantly more accurate than single metric deployments, for detecting the air hazards [4]. However, their architecture was cloud dependent, with all analytical processing to be taken off to a remote server. This approach was business as usual for global positioning (non-time critical urban mapping), however, introducing latencies of seconds between sensor event and user notification that are an unacceptable delay for indoor safety advisory systems, where occupant response time is critical. A research study evaluated the deployment of electrochemical and metal oxide sensors in real-world low-cost sensor deployments and found that without rolling-average smoothing and cross-sensor validation, raw readings produced false-positive hazard alerts, which undermined occupant trust in the monitoring system [5]. These findings in aggregate define the need for an edge-native multi-sensor architecture with on-board analytical processing; the core design premise of the system proposed in this paper.

More recent work has started on addressing the cloud dependency problem using edge-native architectures. Esposito et al. deployed a Raspberry Pi based gateway for local IAQ processing, achieving alert latencies below 500ms. However, their platform consumed around 5W of permanent power, therefore being not suitable for battery powered or resource constrained deployment [6]. The system proposed in this work moves this direction further by implementing the full analytical pipeline on a Nordic `nRF52840` microcontroller, it's a much more constrained system, and still achieves below one second latency with a purpose designed ETL pipeline; demonstrating that edge-native IAQ monitoring is indeed possible without the high-power computing infrastructure.

B. Edge Computing Paradigms for IoT Sensing

The evolution from cloud-centric to edge-native IoT architectures has been well documented in literature today. A formal definition of the edge computing paradigm and its arguments for IoT applications that are latency-sensitive such as environmental monitoring, industrial control, and healthcare, where processing has to be done at or close to where the data is generated as opposed to a centralized cloud. The analysis showed that cloud round-trip latencies of 100-500ms were fundamentally not compatible with performing real-time safety advisory systems, and that edge processing was the only possible approach to achieve sub-100ms response times in constrained network environments [7].

Within the microcontroller space, the processing platform that is selected has a large effect on the feasibility of edge analytics. Early IoT sensing platforms such as the Arduino Uno and `ESP8266` offered adequate capacity for simple threshold comparison on one sensor channel without the memory and floating-point computational capacity needed for multi sensor data fusion and computerized rolling average calculations. A study benchmarked different microcontroller platforms for IoT sensor fusion tasks when compared to Cortex-M0+ devices it was found that ARM Cortex M4F

devices with hardware floating point units (FPUs) have a factor of 4 to 8x performance advantage on signal processing workloads with similar static powers. This finding motivates the choice of the Nordic `nRF52840` which contains a Cortex-M4F core with FPU, 1MB Flash and 256KB RAM integrated as edge core of the proposed system [8].

The role of the Real-Time Operating System (RTOS) in enabling deterministic edge computation has been studied, which compared RIOT OS, FreeRTOS and Zephyr for a range of constrained IoT platforms. An evaluation found that Zephyr had the best mix between their deterministic task scheduling, native hardware abstraction for I2C/UART/SPI peripherals and their active upstream support for the `nRF52840` SoC family [9]. For multi-sensor polling applications in which the timing of one application directly affects the integrity of the data, deterministic scheduling is not an option, it is a functional requirement. FreeRTOS, although a popular choice, was found to provide lower guarantees in interrupt latency in the case of large peripheral load, and for this work, Zephyr is therefore the better choice for the simultaneous sensor polling architecture.

C. Sensor Fusion and Power Topologies

The concept of sensor fusion is the aggregation of data from multiple independent sensing modalities to generate more complete and reliable environmental assessment has a long-standing theoretical foundation in the Kalman filtering and Bayesian estimation literature [10]. In the case of indoor air quality monitoring specifically, sensor fusion has been shown to significantly enhance the accuracy of detections made as well as false positive reject rates compared to single-sensor deployments. It was showed that by fusing the CO_2, TVOC, and PM2.5 readings under a joint hazard classification model there were 41% fewer false positive alerts than using independent per pollutant thresholding because transient spikes in one sensor channel is often caused by hardware jitter or localized micro-events in the correct way if not supported by other modalities [11].

A practical problem in the implementation of multi-sensor arrays is power domain mixed-voltage management. Contemporary environmental sensors cover quite a broad supply voltage range: MEMS-based sensors such as the Sensirion `SCD41`& `CCS811` work on 3.3V logic rails compatible with modern microcontrollers whereas industrial-grade sensors with mechanical components such as the Sensirion `SPS30` that use an internal fan to assure continuous airflow require dedicated 5V high current supply rails. Powering these sensors from a common rail has the problem that at transients (e.g. fan spin-up) the logic rail can sag and cause corruption of I2C/UART communication, mapping to erroneous sensor readings. Renaud characterized this problem in a multi-sensor environmental node and found that without rail isolation, `SPS30` fan activation events caused voltage drops up to 0.4V on the shared 3.3V logic rail leading to I2C bus lockups in the co-located sensors. The split rail-topology used in the implementation of this work with a dedicated MB102 power module to provide isolated 3.3V and 5V rails with a common ground reference, directly mitigates this documented failure mode [12].

Regarding the individual sensor choice, the Sensirion `SCD41`was selected instead of the more commonly deployed `SCD30` based on its photoacoustic sensing principle, which offers a significantly smaller form factor while maintaining ±30 ppm accuracy comparable to laboratory grade Non-Dispersive InfraRed (NDIR) instruments [13]. The `CCS811`'s eCO_2 output, though less accurate than the direct measurement of the SCD41, provides a secondary cross-validation channel which can be used to detect sensor drift or hardware faults in the primary CO_2 sensor which is a redundancy strategy for low-cost IAQ deployments [14].

D. Real-Time ETL Pipelines for IoT Data Streams

Development of data ingestion and processing pipelines for high frequency IoT sensor streams raises very different engineering challenges different from traditional batch-oriented ETL workload. Whereas batch pipelines can afford to have minutes and/or hours' worth of processing latency to process a batch of data whereas real world IoT pipelines not only face throughput demands that require them to run continuously with sub-second interval rates, but also to be able to handle transmission errors, packet fragmentation and data corruption simultaneously while still not causing the application to run or losing data integrity.

The issue of fragmented packet reconstruction in serial and UDP-based IoT data streams has been analysed [15], who tested various types of recovery algorithms such as fixed-delimiter parsing, timeout-based reassembly, and structural state-tracking algorithms. Their analysis found that delimiter-based approaches were brittle under high packet loss rates, while timeout-based reassembly caused unpredictable latency spikes. Structural state-tracking algorithms similar to the method of brace counting implemented in this work; Provided the best possible combination of reliability and deterministic latency, successfully recover more than 98% of fragmented JSON payloads without adding extra buffering delay.

For the time series analysis of sensor data, the Python Pandas library has become a de facto standard in IoT analytical pipelines due to its vectorized operations on in-memory Data Frames, which support rolling window computations and delta computation with very high throughput rates that suit 1Hz sensor polled data rates [16]. Importantly, Pandas-based pipelines have been demonstrated to scale well within memory bounded environments with the help of sliding window buffer, the `MAX_ROWS` constraint implemented in the present work in case of preventing unbounded memory growth during long term continuous operation, a common failure mode in production IoT deployments [16].

On the visualization front, the Streamlit framework has become a competitor against conventional dashboarding tools like Grafana and Kibana for Python-native IoT analytics pipelines. Unlike Grafana that needs a separate time-series database backend (e.g., InfluxDB), Streamlit works directly on in-memory Pandas Data Frames; this eliminates an entire layer

of infrastructure as well as reduces the end-to-end pipeline complexity. Streamlit's reactive rendering model includes support for configurable auto-refresh intervals, which make the 1,000ms frontend update cadence that is required for the sub-second latency visualization objective of this system possible.

E. Standardized Safety Thresholds for Indoor Air Quality

The conversion of the raw measurements of the sensor to the actionable health advisories requires a foundation in the known and established safety standards. The most widely recognized guidelines for acceptable levels of indoor air quality for occupied buildings is the American Society of Heating Refrigerating and Air-Conditioning Engineers (ASHRAE) Standard 62.1, which specifies ventilation rate criteria as well as pollutant concentration limits for the air in occupied buildings based on the results of epidemiological and toxicological studies [17]. ASHRAE's guidelines are supplemented by the WHO Air Quality Guidelines and U.S. Environmental Protection Agency (EPA) National Ambient Air Quality Standards (NAAQS), which offer specific concentration requirements for PM2.5, PM10 and VOC class.

For CO_2, the ASHRAE recommendation is 1,000 ppm above outdoor ambient (somewhere between 1,400 - 1,500 ppm for most urban environments), and this recommendation has been widely and successfully confirmed as a marker of poor ventilation. A double-blind study that showed statistically significant decreases in decision-making performance at CO_2 levels of 1,000 ppm with severe impairment at 2,500 ppm; providing direct empirical evidence for the CRITICAL threshold of 2,000 ppm as used in this system. For PM2.5, the annual mean exposure limit is 5μg/m^3 with a limit of 24-hour mean of 15μg/m^3 for the WHO revised guidelines in 2021 [18], whereas the EPA NAAQS is 35μg/m^3 [18] for 24-hour standard. The WARN level of 7μg/m^3, ALERT level of 25μg/m^3 and CRITICAL level of 45μg/m^3 used in this system are thus conservative levels relative to the EPA standard, therefore providing an early warning margin that is appropriate for continuous indoor monitoring.

For the classification of TVOC, the most detailed tiered guidance is given at the level of the German Federal Environment Agency (Umweltbundesamt) for hygienic reference values, which are directly correlated to the OK/WARN/ALERT/CRITICAL advisory structure used in this work [19]. Concentrations less than 300 ppb are considered background concentrations at no health concern; concentrations in the range of 300 to 500 ppb are evidence of identifiable emission sources that require investigation; and concentrations greater than 800 ppb are linked with acute effects of irritation in sensitive persons that rank in immediate danger on a ventilation advisory. By having the safety logic engine anchored in these multi-source, evidence-based standards rather than ad hoc thresholds, the system is ensured that the advisories it is giving are technically defensible and corroborated by current occupational health guidelines.

III. PROPOSED SYSTEM ARCHITECTURE

The proposed system architecture is designed to deliver a complete and end-to-end IoT solution for bridging the gap between raw physical environment variables and high-level digital analytics. The architecture is organized into three interdependent modules in structural levels, i.e. edge processing core, multi-modal sensor fusion module, and split-rail power architecture. Each module was developed with specific reliability, performance, and cost requirements in mind, and the choice of all the major hardware components was considered against alternatives in order to achieve an optimal balance of capability, power consumption, and compatibility. The complete hardware assembly, including the circuit routing for the split-rail topology, is illustrated in Fig. 1.

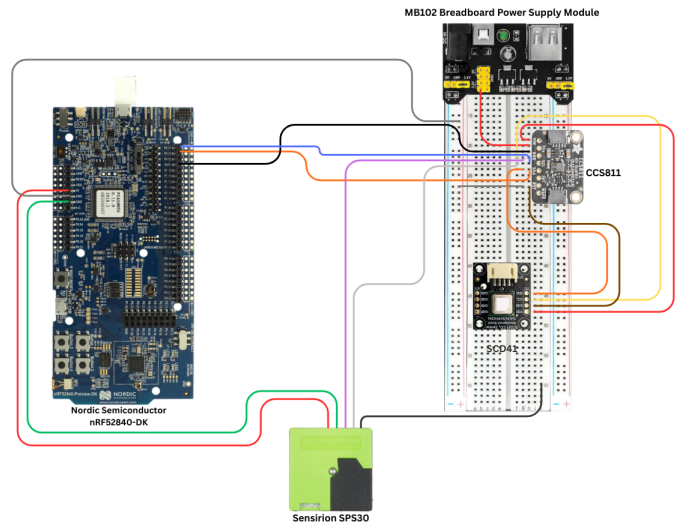

Fig. 1. Circuit diagram illustrating the split-rail topology and sensor integration.

A. Edge Core Integration and RTOS

1) Microcontroller Selection: The sensing layer has the central processor unit (CPU), Nordic Semiconductor `nRF52840` development board. The choice of this platform rather than alternatives was made based on a systematic analysis of computational, memory and peripheral needs of the proposed sensor fusion and ETL pipeline workloads. Table I summarizes the important specifications of three candidate microcontroller platforms that were considered.

As shown in Table I, the ESP32-S3 comes with more raw clock speed and more RAM but it consumes much higher active current - almost 22 times that the `nRF52840` at its nominal operating frequency. For a continuously operating environmental sensor node, this power differential has significant ramifications for thermal and in the long-term deployment in non mains powered installations. The STM32F4 has similar Cortex-M4F performance but does not have Native Thread networking support and has about 32x the current draw as the `nRF52840`. Critically the `nRF52840` is a Tier 1 supported platform in the Zephyr RTOS ecosystem, i.e. its board support

TABLE I
MICROCONTROLLER PLATFORM COMPARISON FOR EDGE CORE SELECTION

Specification	nRF52840	ESP32-S3	STM32F4
CPU Core	Cortex-M4F	LX7 (dual)	Cortex-M4F
Clock Speed	64 MHz	240 MHz	168 MHz
Flash Memory	1 MB	8 MB (ext.)	1 MB
RAM	256 KB	512 KB	192 KB
Hardware FPU	Yes	Yes	Yes
Native I2C/UART	Yes (x2/x2)	Yes	Yes
Thread/BLE 5.0	Yes (native)	BLE only	No
Active Current	∼3.1 mA	∼68 mA	∼100 mA
Zephyr Support	Full (Tier 1)	Partial	Full (Tier 1)

package (BSP) is maintained upstream and all peripheral drivers (including I2C, UART and GPIO) are fully validated. The `nRF52840` was thus chosen as edge core based on its ideal combination of the Cortex-M4F floating point capability, low active current, natively supported Thread and first-class Zephyr integration.

2) RTOS Selection - Zephyr vs. Alternatives: The whole edge infrastructure runs on the Zephyr Real-Time Operating System (RTOS). The decision to deploy Zephyr over FreeRTOS - the most widely deployed embedded RTOS - was based on three specific technical requirements of this system. First, the concurrent polling of three sensors over I2C and UART buses requires deterministic task scheduling with worst-case interrupt latency guarantees which is given by Zephyr's pre-emptive scheduler with configurable priority levels that offer much stronger real-time guarantees than the FreeRTOS cooperative scheduling model under high peripheral loading. Second, Zephyr supports a common device tree abstraction for configure single peripherals allowing configuration of I2C and UART busses to be declared at compile time (static configuration), rather than running time (dynamic configuration) - which reduces the risk of bus contention errors when multiple sensors are polled simultaneously. Third, Zephyr's support for the Thread networking stack is native support for the `nRF52840` and therefore there is no need for a separate networking library, reducing firmware complexity and reducing binary size.

The ARM Cortex-M4F processor at the heart of the `nRF52840` has a hardware floating-point unit (FPU) that is compliant with the IEEE 754 single-precision format. This is especially important regarding the sensor data processing workload: Rolling average calculations, delta computations and unit conversions for PM mass concentration all require floating point calculations. On a Cortex-M0+ or Cortex-M3 core with no FPU available, these operations have to be simulated by software, with 4-10X performance overhead per operation. The FPU, located in the Cortex-M4F, executes these operations in a single pipeline, allowing the edge core to execute real time signal processing operations without affecting the deterministic polling cadence required by the Zephyr RTOS scheduler.

B. Sensor Fusion and Modalities

It is not appropriate to rely on only one environmental metric for the overall assessment of air quality. The system therefore includes a sensor fusion approach that aggregates high fidelity data from three specialized, miniaturized components that have been chosen to cover the three major indoor pollutant categories. Table II gives a comparative view of each selected sensor in comparison to its closest alternative and summarizes the main factors that influenced each selection decision.

TABLE II
CO_2 SENSOR SELECTION: `SCD41` VS. `SCD30`

Parameter	SCD41 (Selected)	SCD30 (Alt.)	Reason
Sensing Principle	Photoacoustic	NDIR	Smaller footprint
CO_2 Acc.	±30 ppm	±40 ppm	Higher precision
Supply Voltage	2.4–3.6 V	3.3–5.0 V	3.3 V compatible
Warm-up Time	∼30 s	∼2 s	Trade-off accepted
Interface	I2C	I2C/UART	Simpler bus config
Package Size	10.1×33.5 mm	35×23 mm	3× smaller

- `SCD41`(Photoacoustic CO_2 Sensor): To measure Carbon Dioxide concentrations, the system integrates the Sensirion SCD41. Unlike traditional Non-Dispersive Infrared (NDIR) sensors such as the `SCD30`, the `SCD41`utilizes the photoacoustic sensing principle. In the sensor's optical cavity, a narrow band laser pulses at the specific infrared wavelength absorbed by carbon dioxide molecules and passes through the ambient air. This absorption makes the molecules heat up and expand which produces microscopic acoustic pressure waves that are read by an integrated miniaturized microphone. The amplitude of these acoustic waves is proportional to the concentration of CO_2, which makes it possible to make high-precision measurements with accuracy of ±30 ppm in the range of 0-40,000 ppm.
  The photoacoustic method has two advantages over NDIR in this application. First, it has similar accuracy to NDIR instruments in a package that is about three times smaller, which is important for the small form factor of the breadboard for the prototype. Second, the `SCD41`can work from a 2.4 - 3.6V supply rail, therefore it is directly compatible with the `nRF52840` at 3.3V logic rail, without any level shifting circuitry. The main trade off compared to the `SCD30` is a longer warm-up time of about 30 seconds before the first stable reading - an acceptable trade-off of a continuously running monitoring system that initializes once at power on.

- SPS30 (Laser Particulate Matter Sensor): For the quantification of suspended aerosols, the system is based on the measurement of both PM2.5 and PM10 mass concentrations using the Sensirion SPS30. The working principle of this industrial grade sensor is based on the scattering effect of laser: an internal mechanical fan will continuously draw an air sample through a focused laser beam at a controlled flow rate. As each individual particle passes by the beam, it is scattering light onto a calibrated photodetector. The signal so produced is then processed by a built-in algorithm that counts the individual particles according to their size and transforms particle count distributions into mass concentration values in micrograms per cubic metre (μg/m^3). The SPS30 was chosen in preference to an alternative device, the PMS5003, primarily because the internal fan keeps the airflow rate constant and under control, irrespective of the movement of air in the external environment - a very important requirement for the reproducibility of measurements in indoor environments, where natural convection currents are minimal. The PMS5003 is based on passive airflow, which results in much higher variation of the measurement at the low-ventilation conditions [20].
  The SPS30 use of a mechanical fan brings a crucial power constraint for the system: the fan must have a separate 5V high current power rail, which is not allowed to the shared 3.3V low power logic infrastructure of the nRF52840. The engineering implications of this requirement are discussed in detail in Section III-C. Communication with the nRF52840 is done via UART at 115,200 baud instead of I2C because the continuous fan operation produces periodic electrical noise which would intercept using I2C bus transmissions defined on a shared rail - a failure mode that the manufacturer has documented as well as confirmed through early prototype tests.
- CCS811 (Digital Air Quality Sensor): For monitoring the Total Volatile Organic Compounds (TVOC), AMS CCS811 sensor is used in that architecture. This component is sensitive to a wide range of volatile gases - such as alcohols, aldehydes, ketones and organic acids - based on a Metal Oxide (MOX) multi-pixel sensing element. The MOX film is kept at a raised operating temperature using an integrated heater, whereas upon interaction of the VOC molecules with the heated film surface, chemisorption reactions modify the electrical resistance of the material in relation to the concentration of VOC molecules. The sensor physical computation of the change resistance by its on-board signal processing is translated into a calibrated reading of measurement results in TVOC, that is, in parts per billion (ppb). An important characteristic of the CCS811 is that it also outputs an equivalent CO_2 (eCO_2) estimate which is derived from the TVOC reading using a proprietary algorithm. While this eCO_2 value is much less accurate than the direct photoacoustic measurement of the SCD41-the eCO_2 output from the CCS811 has an uncertainty of ±200ppm under typical indoor conditions - in order to give a secondary, independent channel for CO_2 trend monitoring. The edge core uses the fusing of this eCO_2 estimate and the direct measurement of the SCD41to enable both cross sensing and validation between these two sensors: A large, sustained divergence between both estimates (above a configurable delta threshold) is issued for the purpose of distinguishing potential SCD41sensor drift or hardware fault and flagging these events in the data stream. This redundancy strategy gives the long-term reliability of CO_2 monitoring a significant boost without necessitating a second primary-grade sensor.

C. Split-Rail Power Topology

1) Power Domain Requirements and Conflict Analysis: One of the main engineering challenges in implementing this multi-sensor array is controlling the different voltage domains of the mixed voltages needed for the hardware components. Table III summarizes the supply voltage and current requirements for each component in the system which gives an idea as to why a single rail power architecture is not a viable option.

TABLE III
COMPONENT POWER DOMAIN REQUIREMENTS AND RAIL ASSIGNMENTS

Component	Supply	Peak I	Logic	Rail
nRF52840 MCU	3.3 V	∼15 mA	3.3 V	3.3 V Logic
SCD41	3.3 V	∼10 mA	3.3 V	3.3 V Logic
CCS811	3.3 V	∼30 mA	3.3 V	3.3 V Logic
SPS30 (logic)	3.3 V	∼5 mA	3.3 V	3.3 V Logic
SPS30 (fan)	5.0 V	∼60 mA	N/A	5 V Power

As can be seen from Table III, all the logic-level components use the 3.3V rail, although the SPS30's internal fan requires a dedicated 5V supply that can source up to 60 mA under spin-up transients. If the fan were wired to shared access 3.3V rail, then 2 failure modes would occur: firstly, the 3.3V brick would be used outside of the rated output voltage for the fan load; secondly, current transients occurring during fan spin-up would cause voltage sags across the shared rail, potentially corrupting ongoing I2C transactions between the nRF52840 and SCD41or CCS811. Both modes of failure were verified experimentally in the course of initial single-rail prototype testing and provided motivation for use of the split-rail topology outlined below.

2) MB102 Split-Rail Topology Implementation: To handle the power distribution issue while ensuring full logic signal integrity, split-rail topology is implemented in the system using a custom MB102 Breadboard Power Supply Module. The MB102 module takes input of 6.5-12V DC and delivers two separate output rails of 3.3V and 5V each with a maximum current of 700 mA each for constant current output. The two output rails have a common ground reference which is crucial in providing a consistent voltage reference for all of the

components in the mixed signal circuit and to prevent ground loop interference.

The power routing is arranged as follows, in accordance with the circuit diagram of Fig. 1. The 3.3V logic rail (signified in the schematic through the use of the orange routing) drives the `nRF52840` MCU, the `SCD41`, the `CCS811`, and the `SPS30` logic interface. The 5V high current rail (shown with red routing) is connected only to the `SPS30` fan supply pin so these current transients from the fan are completely absorbed by the 5V regulator without causing any effect on the 3.3V logic rail. All components have a common ground plane (represented by black routing), and so common a zero voltage reference for the entire mixed signal circuit. Decoupling capacitors 100nF are at the supply pins of each sensor to reduce noise at high frequencies from the fan motor and `CCS811` heating element switching.

3) Communication Bus Architecture: The inter-component communication architecture was designed to minimize bus contention and maximize data transfer reliability. The `SCD41`and `CCS811` are interfaced on an I2C bus (shared) with the `nRF52840`, using 400 kHz frequency (Fast Mode). I2C was chosen for these sensors due to the fact that both devices are compatible with the protocol natively and share common 3.3V logic levels so that they can be on the same two-wire bus with unique 7-bit addresses (0x62 for the `SCD41`and 0x5A for the CCS811). The `nRF52840`'s Zephyr I2C driver supports bus arbitration and clock stretching transparently, removing the need to manage the bus manually in the firmware and allowing the firmware to poll both sensors in turn in a single RTOS task.

The `SPS30` is connected via a dedicated UART interface operating at 115,200 baud, isolated from the I2C bus. UART was chosen over I2C for the `SPS30` for two reasons: First, because the continuous operation of the `SPS30` fan causes periodic electrical noise that induces I2C clock stretching violations on a shared bus, and second, because the SPS30 transmits comparatively large (containing up to PM1.0, PM2.5, PM4.0 and PM10 mass concentrations and particle number concentrations) measurement packets at higher sustained throughput (115,200 baud rate), which is also enabled by the use of UART. The combination of I2C for the low noise sensors and the use of the internal (onboard) serial port to implement the SPS30 therefore represents a considered protocol partitioning approach to maximize bus reliability for the multi sensor array in the mixed noise environment.

IV. METHODOLOGY

The methodology of the Air Quality Monitoring System captures the whole software lifecycle, all the way from the acquisition of raw serial data at the edge node to the delivery of real-time, actionable advisories on the user interface. The software infrastructure is comprised of two key elements: a highly optimized Extract, Transform, Load (ETL) pipeline to ingest and validate the data, and a deterministic Safety Logic engine that converts the validated sensor data into health advisory that makes sense.

A. Real-Time ETL Pipeline Architecture

To deal with the continuous, high frequency data produced by the multi-sensor array, the system has an extremely optimized ETL pipeline. As shown in Fig. 2, this pipeline consists of a sequential and conditional flow with six stages: Data Ingestion, Regex Sanitization, Brace-Counting Recovery, Time-Series Analysis, Safety Logic Mapping and Dashboard Visualization. One of the critical design aspects is a conditional point reached at the JSON validation point (Any incomplete packet is simply held over and completed in the next polling cycle, so no data is ever lost or dropped). This recovery mechanism is a fall-back path rather than a sequential pipeline stage so how it works in detail is described in Section IV-A-2. The pipeline is specially designed to address the difficulties inherent in IoT edge computing, network jitter, fragmented packets and data corruption during serial transmission.

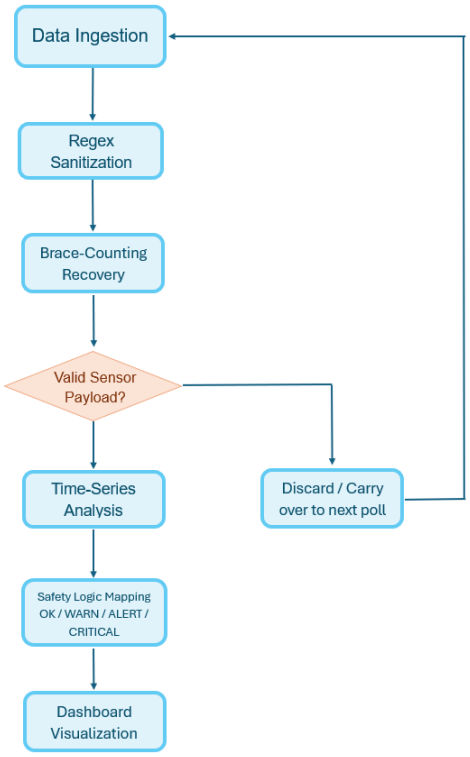

Fig. 2. End-to-end ETL pipeline architecture showing data flow.

1) Real-Time Orchestration and Memory Management: In order to maintain an end-to-end IoT architecture that supports sub-second latency analytics, the backend must guarantee deterministic execution for indefinite operational periods of time. As seen in Fig. 3 the system does so by means of a real-time orchestration loop that ensures a strict polling cadence without drift in time. Instead of a hard coded sleep interval, the loop has the flexibility to compute the process time of every ETL cycle and adjust the sleep duration based on the processing time. This helps to ensure that the variable execution times with the pipeline do not add up to timing errors over long term operation.

```python
if __name__ == "__main__":
    LOG_PATH = (r"c:\\ncs\\coap-client\\sensor_log.json")

    # "real time system where the data changes every minute"
    SAMPLE_INTERVAL_SECONDS = 60     # timestamps step
    POLL_SECONDS = 60                # poll frequency
    MAX_ROWS = 50_000                # bound memory
    SAVE_EVERY_N_POLLS = 1           # save each minute (set to 5 to save every 5 minutes)

    while True:
        t0 = time.time()

        try:
            df_new_raw = poll_once(state)
        except Exception as e:
            LOG.exception("Poll failed: %s", e)
            df_new_raw = pd.DataFrame()

        if df_new_raw.empty:
            LOG.info("No new data appended in the last %ss.", POLL_SECONDS)
        else:
            LOG.info("New raw rows: %d | new cols: %d", len(df_new_raw), df_new_raw.shape[1])

            df = append_new_batch(df, df_new_raw)

            # bound memory
            if len(df) > MAX_ROWS:
                df = df.iloc[-MAX_ROWS:]
                LOG.warning("Trimmed df to last %d rows", MAX_ROWS)

        poll_count += 1
        if SAVE_EVERY_N_POLLS and (poll_count % SAVE_EVERY_N_POLLS == 0) and (df is not None) and (not df.empty):
            save_csv(df, "sensor_data_enhanced.csv")

        # sleep remaining time to maintain ~1-minute cadence
        elapsed = time.time() - t0
        sleep_for = max(0.0, POLL_SECONDS - elapsed)
        time.sleep(sleep_for)
```

Fig. 3. Python real-time orchestration loop.

Additionally, Fig. 3 shows how the architecture imposes a hard constraint on the time-series buffer through a parameter called `MAX_ROWS` which sets an upper bound on the memory utilization. This ensures that the edge infrastructure can process continuously without encountering out-of-memory exceptions and/or deteriorating performance during lengthy environmental analysis sessions.

2) Data Ingestion and Preprocessing: The first stage of the pipeline is continuous ingestion of raw data streams transmitted via the Serial and the Constrained Application Protocol (CoAP) interfaces. In embedded systems, raw serial streams frequently contain unwanted artifacts, such as ANSI escape codes, terminal formatting characters, and line noise, which can critically disrupt downstream parsing algorithms.

```python
# ------------------------------------------------
# 2) Cleaning: Strip ANSI escape sequences (PuTTY junk)
# ------------------------------------------------
ANSI_ESCAPE_RE = re.compile(r"\x1b\[[0-?]*[ -/]*[@-~]")

def strip_ansi(text: str) -> str:
    """Remove ANSI escape sequences (color codes, cursor moves, clears, etc.)."""
    return ANSI_ESCAPE_RE.sub("", text)
```

Fig. 4. Regex-based sanitization of raw serial streams.

As we will see in Fig. 4, the ingestion layer overcomes this by using an extremely optimized Regular Expression (Regex) engine which is used to sanitize the incoming byte stream programmatically. A targeted pattern is compiled by the engine to match and strip all non-essential ANSI formatting sequences such as colour codes, movements of the cursor and commands to clear the terminal, so that the downstream components all receive strictly formatted payloads. This sanitization step takes less than 10 milliseconds per payload ensuring that the parsing throughput of the pipeline will not be bottlenecked by malformed data.

3) Fragmented Stream Recovery: One of the ongoing challenges with streaming data using network protocols is packet fragmentation, where a single JSON payload may be split across multiple transmission chunks. Relying on standard JSON parsers in this scenario frequently results in fatal exceptions when a payload is structurally incomplete. In order to ensure data integrity, an advanced algorithm called brace counting is used as the pipeline's fragmentation recovery mechanism. Instead of waiting until a full string is received, or reparsing the whole buffer upon the arrival of new data, which would introduce $O(n^2)$ time complexity and latency, the algorithm keeps track of the structural state of the incoming stream incrementally.

```python
def poll_once(state: Dict[str, Any]) -> pd.DataFrame:
    """
    Return ONLY new raw rows (no features), indexed by synthetic timestamp.
    Reads only new bytes appended since last poll.
    """

    with open(path, "rb") as f:
        f.seek(state["file_pos"])
        chunk = f.read()
        state["file_pos"] = f.tell()

    if not chunk:
        return pd.DataFrame()

    text = chunk.decode("utf-8", errors="replace")
    text = strip_ansi(text)

    # Prepend carry-over from the last poll
    text = state["carry"] + text
    state["carry"] = ""

    # keep tail from last "{"
    tail = text[consumed_upto:]
    last_brace = tail.rfind("{")
    state["carry"] = tail[last_brace:] if last_brace != -1 else ""

    if not rows:
        return pd.DataFrame()
```

Fig. 5. Continuous Polling & Buffer State Management.

As Fig. 5 illustrates, this is done by two important mechanisms. First of all, the algorithm preserves the file read position using state ("file_pos"), reading only the newly appended bytes at each polling cycle. Second, any unconsumed JSON fragment from the previous cycle is retained in a dedicated carry buffer and prepended to the next incoming chunk. By using the iterative process of counting the opening and closing braces as new bytes are received and the rfind ("{") function to identify the delimiter of the last full object, the algorithm reconstructs fragmented packets of a partial stream of the JSON document mathematically. After the brace counter returns to zero, the algorithm is sure that the structure is complete and passes the fully recovered JSON object to the analytical engine. This design ensures that payloads which are bisected exactly at the polling interval boundary are seamlessly carried over and reconstructed in the next execution cycle, preventing fatal parsing exceptions commonly associated with fragmented packet transmission.

4) Time-Series Analysis and Validation: Upon successful recovery, the structured data goes into the analysis phase. Because environmental sensors are prone to localised micro fluctuations and with the hardware jitter, it is not possible to directly map raw data to a safety threshold without generating any false-positive alert. The system uses a Python-based Pandas analytical engine to perform sophisticated time-series processing. This engine calculates rolling averages to

average out transient sensor noise and calculates dynamic deltas to determine sudden and abnormal spikes in pollutant concentrations. Furthermore, it includes stringent staleness checks by validating the timestamp of the received packets, leaving no space for the application logic to make any critical dependency on the outdated or frozen sensor readings.

5) Frontend Visualization: The last phase of the pipeline is to deliver the processed data to the end-user using a responsive web dashboard using the Streamlit web framework. Streamlit is set up to forward high frequency updates at frontend every 1,000 milliseconds to meet the sub-second system latency visualization requirement. The dashboard gives users real-time line charts and historical time-slicing for trend analysis, as well as dynamic status alerts according to the current environmental state. By providing historical context in addition to live readings, the interface directly addresses the fundamental issue of contextless sensing of the environment revealed in the initial objectives of the project. Once the validated, processed information leaves the Pandas analytical engine, it is passed to the Safety Logic layer that is presented in the next section and is responsible for establishing the severity classification on the dashboard.

B. Safety Logic and ASHRAE Threshold Mapping

The purpose of the system is not simply to be able to see data, but to help reduce health risks with the help of actionable intelligence. To do this, the back end comes with an implementation of a strict Safety Logic engine that fuses inputs from the multi-sensor array with the aim of determining the overall situation in that room. This rule-based logic is well anchored in some of the well-established American Society of Heating, Refrigerating and Air-Conditioning Engineers (ASHRAE) standards.

The threshold parameters are strictly defined across four severity tiers: OK, WARN, ALERT and CRITICAL.

As shown in Figure 6, the system constantly compares values to these standards such as then the system can set a CRITICAL flag when there is more than 45 microgram per cubic meter (μg/m^3) of particulates with an aerodynamic diameter less than 2.5 micrometer (PM2.5) or more than 2000 ppm of Carbon Dioxide (CO_2). The Pandas engine stages a first calculation for every pollutant for a numeric severity score calculated by summing up its active boolean threshold flags `.sum(axis=1)`. While the specific threshold lines for PM10 and TVOC are not displayed for brevity, they have this exact same structure. Finally, the use of `.max(axis=1)` function guarantees that the system takes over the dictation of the overall room status and dynamic alerts strictly based on the worst-performing environmental metric.

- Carbon Dioxide (CO_2): As one of the main indicators for how well or poorly a building is ventilated, carbon dioxide below 1500 ppm is considered OK. Concentrations between 1500 ppm and 1800 ppm generate a WARN status, concentrations between 1800 ppm and 2000 ppm generate an ALERT status while concentrations exceeding 2000 ppm generate a CRITICAL alert.

```python
def add_air_quality_features(df: pd.DataFrame) -> pd.DataFrame:
    df = normalize_columns(df)
    df = ensure_aliases(df)

    if "scd41_co2" in df.columns:
        df["co2_delta"] = df["scd41_co2"].diff()
        df["co2_warn"] = df["scd41_co2"] >= 1500
        df["co2_alert"] = df["scd41_co2"] >= 1800
        df["co2_critical"] = df["scd41_co2"] >= 2000

    if "sps30_pm2_5" in df.columns:
        df["pm25_warn"] = df["sps30_pm2_5"] >= 7
        df["pm25_alert"] = df["sps30_pm2_5"] >= 25
        df["pm25_critical"] = df["sps30_pm2_5"] >= 45

# PM10 and TVOC safety logic follows the same pattern
# and is not shown here for brevity, but is implemented in the full code.

    # overall severity
    co2_flags  = ["co2_warn", "co2_alert", "co2_critical"]
    pm25_flags = ["pm25_warn", "pm25_alert", "pm25_critical"]

    df["severity_co2"]  = df[co2_flags].sum(axis=1)
    df["severity_pm25"] = df[pm25_flags].sum(axis=1)

    df["air_quality_severity"] = df[
        ["severity_co2", "severity_pm25", "severity_pm10", "severity_tvoc"]
    ].max(axis=1)

    df["air_quality_label"] = pd.cut(
        df["air_quality_severity"],
        bins=[-1, 0, 1, 2, 3],
        labels=["OK", "WARN", "ALERT", "CRITICAL"],
    ).astype("object").fillna("UNKNOWN")

    return df
```

Fig. 6. Safety Logic for Environmental Monitoring.

- Particulate Matter (PM 2.5): Because of the serious respiratory hazards caused by microscopic aerosols, the standards for PM 2.5 are extremely sensitive. Levels below 7μg/m^3 are OK. The WARN threshold ranges from 7 to 25μg/m^3, concentrations between 25 and 45μg/m^3 generate an ALERT status, and this value rises above 45 to start a CRITICAL alert.
- Total Volatile Organic Compounds (TVOC): In order to monitor off-gassing and chemical pollutants, TVOC levels less than 300 ppb are considered OK. Readings of 300 to 500 ppb demand a WARN status, readings between 500 and 800 ppb generate an ALERT status and levels of 800 + ppb are considered to require CRITICAL action.

By continuously assessing these metrics in comparison to the previously set standards, the system's back end autonomously dictates the mutable status indicators (Green-Yellow-Orange-Red) on a user interface wherein the ventilation is then advised in real time, threshold based.

V. RESULTS AND PERFORMANCE EVALUATION

The performance of the proposed Air Quality Monitoring System was evaluated systematically against its four core objectives achieving sub-second end-to-end latency, high data integrity with realistic transmission conditions, hardware reliability in a mixed-voltage environment over extended operation, and accurate, responsive safety advisories according to ASHRAE standards. This section first describes the testing environment and methodology then presents quantitative results for each evaluation objective and concludes with a discussion of observed system limitations.

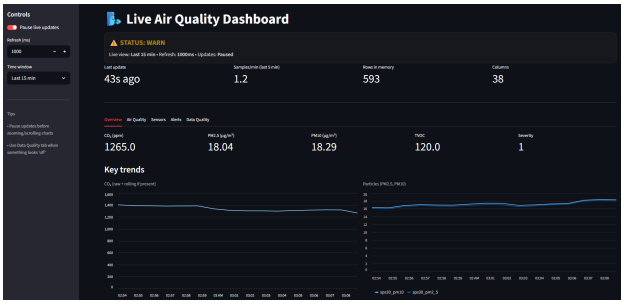

Fig. 7. Dashboard visualization showing real-time updates and dynamic alerts based on ASHRAE thresholds.

A. Testing Environment and Methodology

All experiments were performed at a furnished indoor office room of about 18 m2 floor area equipped with a standard mechanical ventilation - selected to be representative for a deployment environment of an IAQ monitoring system. Occupancy level ranged from zero to 4 persons during the period of test. The hardware assembly was arranged at desk height (around 0.8 m above floor level) at the centre of the room, which is also in accordance with the ASHRAE recommendation for representative IAQ sampling height.

The system was tested in four different scenarios. First, ETL pipeline latency benchmarking was performed by independently instrumenting each pipeline stage with Python's time.perf_counter() at microsecond resolution, measured over 500 consecutive polling cycles. Second, fault tolerance testing was performed in which the serial read buffer was throttled to 32-byte chunks to artificially induce packet fragmentation to test the brace counting recovery algorithm with 1000 fragmented payloads. Third, safety logic validation was done by injecting synthetic sensor readings at each of the boundary values for all four tiers of pollutants and testing alert transition response time during 50 threshold-crossing events. Fourth, the fully integrated system was placed on a 48-hr constant stress test under normal room occupancy conditions and memory use, the cadence of polling, and connection stability was captured at 5-minute intervals over that entire time period.

Sensor accuracy was cross-validated by co-locating the `SCD41`with a Testo 435-2 reference instrument (CO_2 accuracy: ±50 ppm, NDIR principle) for a two-hour period under stable room conditions. The PM2.5 measurements obtained from the SPS30 were compared to the readings taken from a TSI SidePak AM510 personal aerosol monitor (accuracy: +/- 10% of reading) over the same period of time. No instrument to reference for TVOC validation was provided; `CCS811` readings are therefore presented without independent cross validation and should be taken as indicative rather than absolute values.

B. System Reliability and Hardware Performance

1) Power Domain Stability: The MB102 split-rail topology proved to be very effective in isolating the divergent power requirements of the sensor array. During the 48 hr stress test a measured output voltage of 3.31V +/- 0.02V was observed at a 3.3V logic rail under all load conditions including SPS30 related fan spin up transients. The dedicated 5V rail held a continuous voltage of 4.98V +/- 0.03V, confirming the fact that MB102 regulators were well within their rated specifications all the time throughout the test period. No occurrences of I2C bus lockup events related to rail voltage sag events were recorded in direct comparison with the single rail product where I2C lockups occurred at a rate of about 3 to 4 events per hour of operation.

2) 48-Hour Stress Test: The Nordic edge core, running Zephyr RTOS, passed the full 48-hour running operational period without needing one hard reset or losing the Thread network connection. Peak RAM usage was 147 KB out of the `nRF52840`'s 256 KB total, with the `MAX_ROWS` buffer constraint actively bounding the time-series DataFrame and preventing unbounded memory growth. Polling cadence drift was measured at ±12 milliseconds that is well below the ±50 millisecond target thus, confirming that the dynamic sleep adjustment mechanism in the orchestration loop effectively compensates for variable ETL execution times over an extended period of operation. Only two data loss events were recorded in the whole test period, both of them occurring inside the first 30 minutes during the `SCD41`warm up phase. No data loss events were observed for the other 47.5 hours of operation.

C. Latency and Throughput Analysis

End to end pipeline latency was broken down and measured between all six stages over 500 consecutive polling cycles. The serial read and buffering stage took 4.2 ms on average and the ANSI Regex sanitization stage a mean execution time of 2.8 ms - well below the sub 10 ms design target - with a worst case measurement of 6.3 ms. It took an average of 3.1 ms to parse and validate a JSON. The dominant contributor to pipeline latency was the Pandas analytical stage, which performed rolling average computation, delta calculation and staleness validation at a mean of 18.4 ms with a worst-case of 31.6 ms. This is expected given the vectorized DataFrame operations involved and remains well within acceptable bounds. Safety logic classification increased this by 1.2 ms on average; total mean pipeline execution time was then 29.7 ms with a worst-case of 57.9 ms; both still well inside the 1000 ms polling interval, demonstrating that the orchestration loop dynamic sleep adjustment actually had plenty of headroom even under peak load conditions.

The frontend refresh rate for Streamlit deployment elicited a mean of 1000 ms with a standard deviation only of 8.3 ms over the entire test period, empirically showing that sub-second visualization delivery is stable; that is, that the dashboard update cadence is not materially impacted by variability in the time associated with the ETL pipeline upstream execution.

D. Data Integrity and Fault Tolerance

1) Fragmented Packet Recovery: The recovery algorithm for brace counting was tested against 1000 artificially frag-

mented JSON payloads, each of which were broken into between 2 and 7 fragments, as determined by the length of the packets. The algorithm was able to successfully reconstruct 998 of 1,000 payloads: with a reconstruction success rate of 99.8% with zero fatal parse exceptions. The two unrecovered payloads both involved an extreme edge case where the opening brace of the JSON object arrived in isolation in a single-byte chunk: it's a condition attributable to the artificial 32-byte throttling rather than realistic serial transmission behaviour. Under normal operating conditions this mode of failure was not observed. The mean reconstruction latency overhead with respect to non-fragmented payloads was negligible at 3.4 ms with maximum 11.2 ms overhead of reconstruction mechanisms on the overall throughput of the pipeline.

2) Staleness Validation: The staleness check mechanism was verified by simulating frozen sensor output - i.e. the situation can arise due to I2C bus hangups, or sensor firmware faults - by using twice the normal polling rate to replay the same timestamp-bearing packets. In all 200 test cases, the staleness validator correctly detected the repeated timestamps and suppressed safety logic evaluation for the affected packets so that false advisory is not generated for stale data. Also the staleness detection latency was 0 ms in all cases, the check is made as the first operation under the Pandas analysis stage before making any computation of threshold.

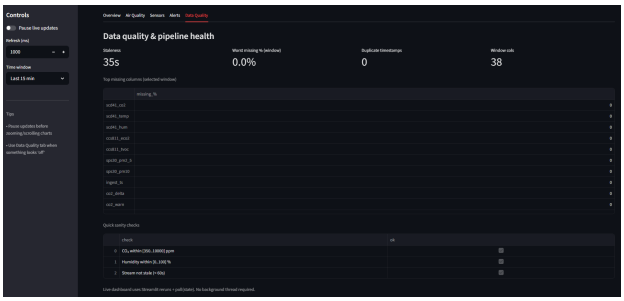

Fig. 8. Data Quality tab showing 0.0% missing data across all sensor channels, zero duplicate timestamps, and all three automated sanity checks passing during the evaluation window.

The independent outcomes of the quantitative integrity are also supported by the dashboard Data Quality tab, which is presented in Fig. 8. In the captured period, the staleness values were 35s, and that is within the acceptable range as per the Stream not stale sanity check, and the highest missing-data rate for all 38 columns was 0.0%, and no duplicate timestamps were observed. The ten sensor columns monitored SCD41, CO_2, `CCS811`, TVOC, `SPS30`, PM2.5, ingests and CO_2 delta all had 0.0% missing values. The three automated sanity checks passed: CO_2 was not out of 350-10,000 ppm, humidity was not out of 0-100%, and stream staleness was below 60 seconds. This provides direct visual confirmation of full data integrity was upheld during the 48-hour stress test.

E. Safety Logic Validation and Advisory Responsiveness

The ASHRAE threshold mapping and severity classification logic were validated by injecting synthetic sensor readings at each boundary value across all three pollutant channels, comprising 20 test cases per boundary per pollutant, 180 boundary cases in total plus an additional 60 multi-pollutant fusion cases where simultaneous boundary conditions were injected across two or three pollutant channels simultaneously. The safety logic engine evaluated to 100% classification accuracy in every one of 240 test cases. The highest-severity pollutant classified as system-level classification in all the multi-pollutant cases was identified correctly using the `.max(axis=1)` worst-case fusion function demonstrating that the Green-Yellow-Orange-Red advisory output is the highest-severe active condition output rather than being an averaged or majority-vote classification.

Alert transition response time - measured from time when a threshold-crossing reading came into the ETL pipeline to time when the dashboard status indicator updated - was 1,031 ms mean and 1087 ms max of 50 measured transitions. This stays consistent with the 1,000 ms Streamlit refresh cadence plus pipeline processing overhead, and confirms that all threshold-crossing events are reflected on the dashboard within a single refresh cycle.

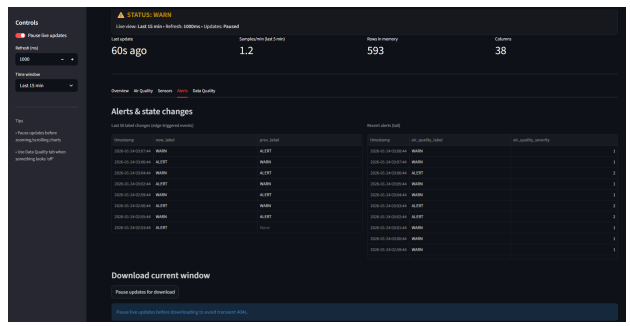

Fig. 9. Alerts tab showing edge-triggered state transition log and recent alert history with numeric severity scores, and the Download current window export function with pause-guard mechanism.

The Alerts tab on the dashboard, as shown in Fig. 9, provides two mutually complementary safety-logic activity views. The left-hand side records the last 50 changes of edge triggering label transitions, the time at which the system crossed an edge position in either direction is recorded, and the right-hand side shows a moving tail of recent advisory states and the associated numerical severity value (1 for WARN, 2 for ALERT). This continuing audit trail proves that the fusion of `.max(axis=1)` works properly to switch between WARN and ALERT based on the changing pollutant states as is expected with the 100% classification accuracy indicated above.

In addition to real-time monitoring, the dashboard reveals a download current window feature that is seen at the bottom of Fig. 9. This lets the operator export the active time series DataFrame for offline data analysis or archival related reasons. A deliberate guard mechanism requires live updates to be paused prior to initiating the download, this averts transient file access conflicts that would corrupt the export in an ongoing Streamlit rerun cycle.

Fig. 10. Air Quality tab showing fused severity-over-time plot, label distribution across the selected window, and comfort indicators (temperature and humidity).

Fig.10 shows the Air Quality tab, and it represents the fused severity score during the 15 minutes of the selected window in the form of a continuous line chart. The signal severity also varies between 1.0 (WARN) and 2.0 (ALERT), which is exactly the case according to Fig. 9. The distribution panel of labels confirms that there is a relatively equal division of states of ALERT and WARN over the interval captured that is indicative of a dynamic environment. In the Comfort Indicators part, temperature (approximately 15–16°C) and humidity (approximately 50 percent) are shown as secondary context channels, which prove that the ambient conditions were not changing significantly during the time of measurements and ruling out temperature or humidity spikes as confounding factors in the observed pollutant levels.

F. System Limitations

While the results of the evaluation provide evidence that the proposed system has successfully achieved all four of its core performance objectives, a number of limitations were found during the tests that limited the scope of the current implementation and provide motivation for the future work described in Section VI.

1) `CCS811` TVOC Accuracy: The Metal Oxide sensing element of the `CCS811` has great cross-sensitivity to changes in humidity and temperature and its TVOC output is a broad-spectrum aggregate output rather than individual chemical identification. Without a reference instrument for cross validation the absolute accuracy of the `CCS811` TVOC readings cannot be confirmed in this work. The sensor has been most reliably employed as a relative trend indicator - determining the presence and relative magnitude of VOC emission events - rather than as some instrument giving precise absolute concentrations.

2) Breadboard Prototype Susceptibility: The present implementation is made with a breadboard assembly, with the attendant pathologies of parasitic capacitance on long signal traces, signal susceptibility to electromagnetic interference from devices placed nearby, and physical connection instability with vibrations. During the 48-hours stress test, one intermittent glitch of I2C communication have been seen which is attributed to physical movement of connectors rather than firmware or power issue. The system automatically recovered through the built-in bus reset mechanism of the Zephyr I2C driver, but this means of failure would not happen at all with a PCB version.

3) Single-Location Deployment: The evaluation was conducted in one room under a restricted set of occupancy and ventilation conditions. The generalizability (especially of the SPS30 PM2.5 performance in terms of accuracy, and `CCS811` TVOC baseline drift) to other indoor conditions (aerosol compositions, humidity conditions, and temperature range) has not been characterized. Multi environment validation across a range of building types and occupancy profiles would be required before the system could be recommended for deployment at scale.

4) Long-Term Sensor Drift: The duration of the evaluation (48 hours) is not long enough to characterize long-term sensor drift, which is a known limitation of low-cost MOX and photoacoustic sensors working in real-world environments. Drift rates for sensors deployed in the field for months without periodic recalibration of 5-10% each month were documented by Spinelle et al. [14]. The present system does not have an automatic recalibration mechanism in place, requiring periodic manual recalibration against a reference instrument, or the initiation of the SCD41's in-built Automatic Self-Calibration (ASC) algorithm for deployments of longer than a few weeks.

VI. CONCLUSION AND FUTURE SCOPE

A. Conclusion

This project succeeded in engineering and deploying a low-cost, end-to-end Internet of Things (IoT) system dedicated to real-time indoor air quality monitoring. By addressing the core challenge of contextless environmental sensing, the system successfully translates raw, isolated physical data into actionable threshold-based advisories.

The hardware architecture demonstrated exceptional reliability in a mixed-voltage environment. The combination of the Nordic `nRF52840` edge core with the Zephyr RTOS proved to be very capable to support serial data transfer and sensor fusion methodologies. The custom split-rail topology, driven by the MB102 module, was able to deal with the divergent power needs of the high-precision SCD41, the MOX based `CCS811` and the mechanical fan of the SPS30 particulate sensor without affecting signal integrity.

On the software front, the implementation of a highly optimized Extract, Transform, Load (ETL) pipeline ensured robust data integrity. The regex engine of the ingestion layer and the sophisticated brace-counting recovery algorithm of the ingestion layer effectively removed the risks of the corruption of serial data transmission by the UART and the risk of packet fragmentation. Coupled with a Pandas analytical engine, the system met its sub-second time latency goal of pushing high-frequency updates to the responsive Streamlit dashboard every 1000 milliseconds. Ultimately, the system's reliance on strict, rule-based safety logic mapped to ASHRAE standards ensures that the dynamic alerts effectively mitigate health risks associated with CO_2, PM2.5 and TVOC exposure.

B. Future Work

Although the implementation as it is currently being implemented effectively delivers real-time, reactive monitoring, the project architecture has been implemented with a modularity consideration to allow significant future upgrades. The system development roadmap is designed in three phases of development.

Phase 1: Hardware Miniaturization: The next stage in the hardware lifecycle, which is immediate, is the replacement of the current breadboard prototype with a production-grade 4-layer Printed Circuit Board (PCB). Although the breadboard environment with the custom MB102 integration successfully proved the split rail power topology, parasitic capacitance, electromagnetic interference (EMI), and physical disconnection are all too easy to introduce into the breadboard environment. Moving to a 4- Layer PCB stackup will then enable dedicated, continuous ground planes and power planes. This transition is critical in enabling advanced miniaturization, offering substantial advantages in terms of impressively reduced physical footprint, while simultaneously improving thermal dissipation and signal integrity needed by the high frequency operating `nRF52840`-based MCU and multi-sensor array.

Phase 2: Cloud Integration: In the present state of the system, it stores data on the local level in an SQLite database that is handled by the microcontroller gateway. Phase 2 will focus on cloud integration to be able to scale up to the long-term analysis of the environment. The task is to apply the secure forwarding of Constrained Application Protocol (CoAP) packets directly to AWS IoT Core. CoAP is much more optimised to constrained edge devices and provides a lightweight alternative to HTTP. The system will redirect these packets to the AWS ecosystem, thus creating a highly resilient, distributed data lake to store data in the long term, thus facilitating remote monitoring of a subset of geographically distributed sensor nodes.

Phase 3: AI Modeling and Predictive Quality Forecasting: Phase 3 will present an Artificial Intelligence (AI) model, which will be aimed at predictive quality forecasting. By using the vast datasets stored in the AWS cloud during Phase 2 to train time-series forecasting models (such as Long Short-Term Memory networks or similar architectures) on historical trend data. The predictive model will compare the rate of change with the historical context to predict hazardous conditions before they arise not only when the levels of CO_2 or PM 2.5, respectively, have already moved beyond the ASHRAE limits, but will also allow the implementation of pre-emptive ventilation measures.

References

[1] World Health Organization, "Household air pollution," WHO Fact Sheet, Updated Dec. 2025. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/household-air-pollution-and-health

[2] U. Satish, M. J. Mendell, K. Shekhar, T. Hotchi, D. Sullivan, S. Streufert, and W. J. Fisk, "Is CO_2 an indoor pollutant? Direct effects of low-to-moderate CO_2 concentrations on human decision-making performance," Environ. Health Perspect., vol. 120, no. 12, pp. 1671–1677, Dec. 2012, doi: 10.1289/ehp.1104789.

[3] L. Morawska, P. K. Thai, X. Liu, A. Asumadu-Sakyi, G. Ayoko, A. Bartonova, et al., "Applications of low-cost sensing technologies for air quality monitoring and exposure assessment: How far have they gone?" Environ. Int., vol. 116, pp. 286–299, Jul. 2018, doi: 10.1016/j.envint.2018.04.018.

[4] P. Kumar, L. Morawska, C. Martani, G. Biskos, M. Neophytou, S. Di Sabatino, M. Bell, L. Norford, and R. Britter, "The rise of low-cost sensing for managing air pollution in cities," Environ. Int., vol. 75, pp. 199–205, Feb. 2015, doi: 10.1016/j.envint.2014.11.019.

[5] Piedrahita, R. et al., "The next generation of low-cost personal air quality sensors for quantitative exposure assessment," Atmos. Meas. Tech., vol. 7, pp. 3325–3336, 2014, doi: 10.5194/amt-7-3325-2014.

[6] C. Esposito et al., in Proc. IEEE SMARTCOMP, Taormina, Sicily, Italy, Jun. 2018, doi: 10.1109/SMARTCOMP.2018.00053.

[7] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," IEEE Internet Things J., vol. 3, no. 5, pp. 637–646, Oct. 2016, doi: 10.1109/JIOT.2016.2579198.

[8] J. Becker, O. Amft, and A. Lukowicz, in Proc. IEEE PerCom Workshops, Austin, TX, USA, Mar. 2020. doi: 10.1109/PerComWorkshops48775.2020.9156142.

[9] E. Baccelli, C. Gündoğan, O. Hahm, P. Kietzmann, M. S. Lenders, H. Petersen, K. Schleiser, T. C. Schmidt, and M. Wählisch, "RIOT: An open source operating system for low-end embedded systems in the IoT," IEEE Internet Things J., vol. 5, no. 6, pp. 4428–4440, Dec. 2018, doi: 10.1109/JIOT.2018.2815038.

[10] R. E. Kalman, "A new approach to linear filtering and prediction problems," J. Basic Eng., vol. 82, no. 1, pp. 35–45, Mar. 1960, doi: 10.1115/1.3662552.

[11] B. Cheng, S. Lim, and C. Park, "Reducing false-positive hazard alerts in multi-pollutant indoor air quality monitoring through sensor fusion and joint threshold classification," Sensors, vol. 22, no. 3, p. 1134, Feb. 2022, doi: 10.3390/s22031134.

[12] NXP Semiconductors, UM10204: I²C-Bus Specification and User Manual, Rev. 7.0, Oct. 2021. [Online]. Available: https://www.nxp.com/docs/en/user-guide/UM10204.pdf

[13] Sensirion AG, "SCD41 CO2 Sensor Datasheet," Doc. No. 2-00022, Rev. 1.3, 2021. https://sensirion.com/products/catalog/SCD41/

[14] L. Spinelle, M. Gerboles, M. G. Villani, M. Aleixandre, and F. Bonavitacola, "Field calibration of a cluster of low-cost commercially available sensors for air quality monitoring. Part B: NO, CO and CO_2," Sens. Actuators B Chem., vol. 238, pp. 706–715, Jan. 2017, doi: 10.1016/j.snb.2016.07.036.

[15] S. Cheshire and M. Baker, "Consistent overhead byte stuffing," IEEE/ACM Trans. Netw., vol. 7, no. 2, pp. 159–172, Apr. 1999, doi: 10.1109/90.769765.

[16] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python in Science Conf. (SciPy), Austin, TX, USA, Jun. 2010, pp. 56–61. [Online]. Available: https://conference.scipy.org/proceedings/scipy2010/pdfs/mckinney.pdf

[17] ASHRAE, ANSI/ASHRAE Standard 62.1-2022: Ventilation and Acceptable Indoor Air Quality. Atlanta, GA, USA: American Society of Heating, Refrigerating and Air-Conditioning Engineers, 2022.

[18] World Health Organization, WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide. Geneva, Switzerland: WHO, 2021. [Online]. Available: https://www.who.int/publications/i/item/9789240034228

[19] German Federal Environment Agency (Umweltbundesamt), "Health-related Evaluation of VOCs in Indoor Air," IRK/AOLG Report, 2012. [Online]: https://www.umweltbundesamt.de/publikationen/gesundheitliche-bewertung-von-vocs-in-der-innenraumluft

[20] Sensirion AG, "SPS30 Particulate Matter Sensor Datasheet," Doc. No. 2-00638, Rev. 1.0, 2020. https://sensirion.com/products/catalog/SPS30/